



Quick Reference

The one-page cheat-sheet — mental models, the file map, and how to read the code at a glance.

The one mental model

- **Frontend** = on the phone (`index.html`). Draws screens, holds a local copy.
- **Backend** = in the cloud (Supabase). The shared source of truth + server jobs.
- Confused? First ask: “frontend or backend?”

The core loop (whole app in 4 steps)

- 1 You tap →
- 2 change the `state` object →
- 3 `save()` (storage + queue sync) →
- 4 `render()` redraws from state.
- *The screen is a function of the data.*

The three web languages

- `<...>` tags = **HTML** (structure)
- `color / padding` = **CSS** (looks)
- `function / if / =` = **JavaScript** (behaviour)

Where things live

- `index.html` — the entire app
- `sw.js` — offline + updates
- `manifest.webmanifest` — “installable app”
- `supabase/functions/` — server programs
- Supabase database — the shared data

Every action's skeleton

- read input → change `state` → `save()` → `render()`
- Add, complete, postpone, edit, delete — all identical.
- Trace one (e.g. `confirmComplete`) and you know them all.

GitHub — two hats

- **Git/GitHub** = history (commits, branches).
- **GitHub Pages** = free hosting of the app.
- Deploy = edit → commit → push → live in ~1 min.

Supabase in one card

- A rented **backend**: database + realtime + functions.
- One table `sparkle_state`, one row per household, whole app as JSON.
- **anon key** = public (not a password). **service key** = secret, server-only.
- Security lives in the database (RLS), never in the app.

How sync stays safe

- Newest change wins per task (`mod` time).
- History = append-only. Deletions leave a **tombstone**.
- Saves use “only if nobody changed it first” (the `rev` counter).
- **Offline-first**: save locally now, sync later.

Reading code like a reviewer — the flags

✓ GREEN FLAGS (CAREFUL CODE)	⚠ RED FLAGS (BE SUSPICIOUS)
Clear names (<code>confirmComplete</code>) <code>if (!x) return;</code> guards at the top <code>esc(...)</code> around user text before display <code>try/catch</code> around network calls Small, single-purpose functions Comments explaining <i>why</i>	User input on screen with no <code>esc</code> (XSS risk) Network calls with no error handling The same code copy-pasted many times Unexplained “magic numbers” One function hundreds of lines long Tricky code with no test

THE HABIT THAT MATTERS MOST

Keep the app **working at every step**, make **small changes**, and **verify each one** (ideally with a test). Ask of any new feature: “How do we know it works?” and “Is any secret shipping to the phone?”

Mini-glossary

state — the object holding all current data	render — draw the screen from state
JSON — universal text format for data	localStorage — per-device browser storage
PWA — a website installable as an app	service worker — background script (offline/updates)
commit / push — a saved snapshot / upload it	deploy — publish a new version
RLS — database access rules	Edge Function — a small server program
tombstone — record that an item was deleted	XSS / escaping — input security bug / its fix